TP 1

CAO Anh Quan
M2 D&K Machine Learning

September 29, 2018

Contents

1 Project Structure

- File **kmeans.R** Contain R code for k-means.
- Folder **audio1, audio2, audio3**: Contain the data about each audio file including:
  - Files with textual integers.
  - Figure of the signal.
  - RAW binary file.
  - Compressed file.

- Folder **k5, k7, k15, k25**: Contain results (integer file, figure, binary and compressed file) after running k-means with the corresponding k value.

- Folder **data**: Some test data for testing the k-means implementation.
- Folder **images**: Some test results from the k-means.

2 Implement k-means algorithm in R

2.1 Implementation

I implement k-means algorithm with 2 initialization modes. We can change the initialization mode by set the **initializationMode** to **random** or **"kmeans++"**.

- **Random**: this is the default mode.

  Choose randomly k points as initial cluster centers.

- **K-means++**:

  Choose randomly the first cluster center from data points. After that, we choose each subsequent cluster center from the remaining observations with probability proportional to the distance between it and the closest existing cluster center.

2.2 Test the k-means implementation

2.2.1 Test 1: 3 separate clusters

Dataset

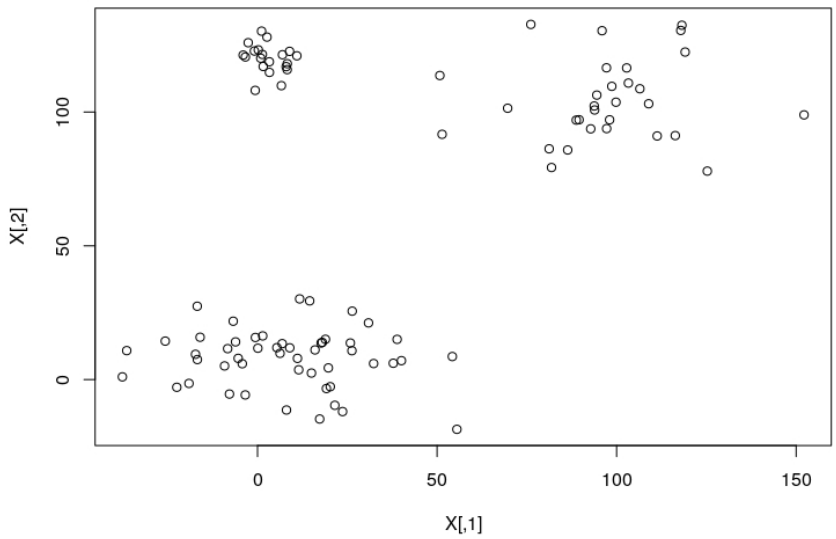
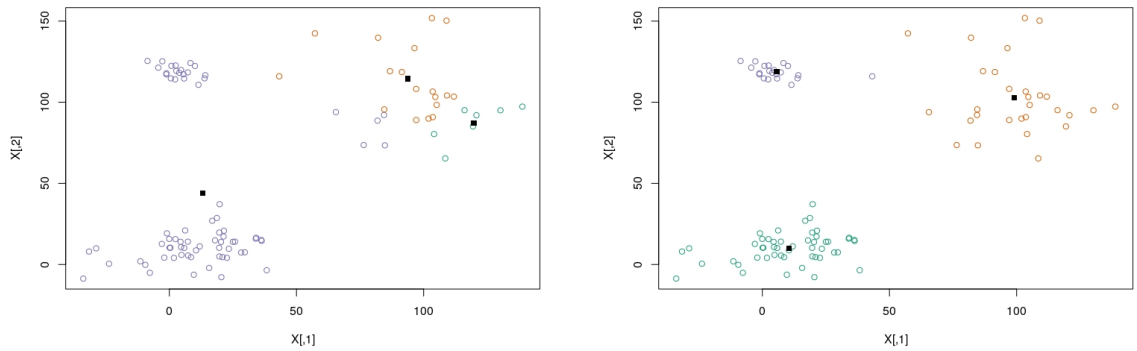


Figure 1: Dataset with 3 clusters

Result



(a) K-means random initialization

(b) K-means++ initialization

2.2.2 Test 2: 3 clusters partially overlap

Dataset

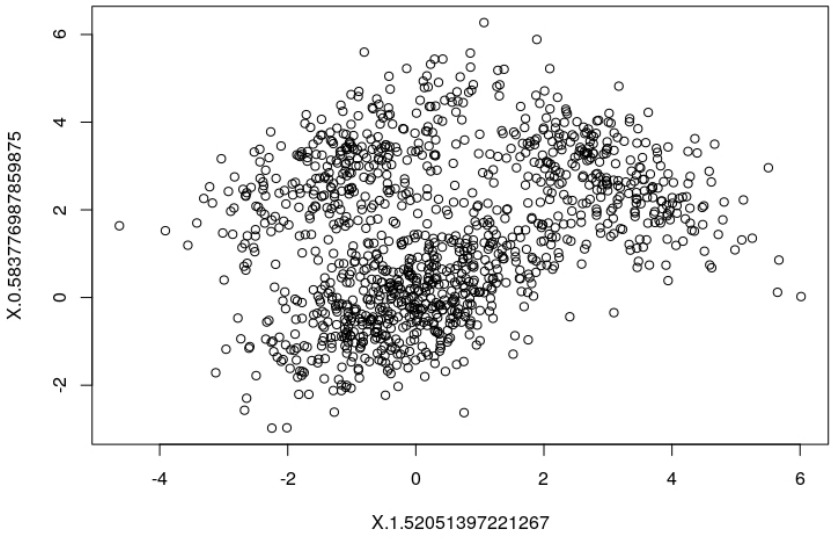
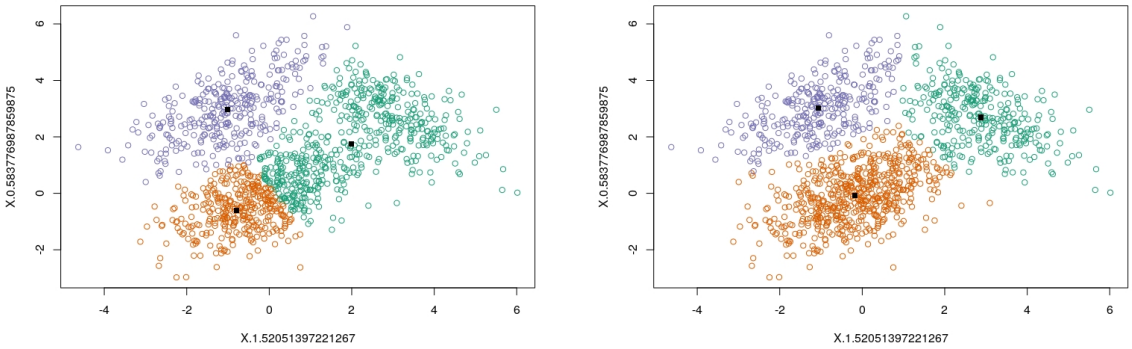


Figure 3: Dataset with 3 clusters partially overlap

Result



(a) K-means random initialization (b) K-means++ initialization

2.2.3 Test 3: 6 clusters partially overlap

Dataset

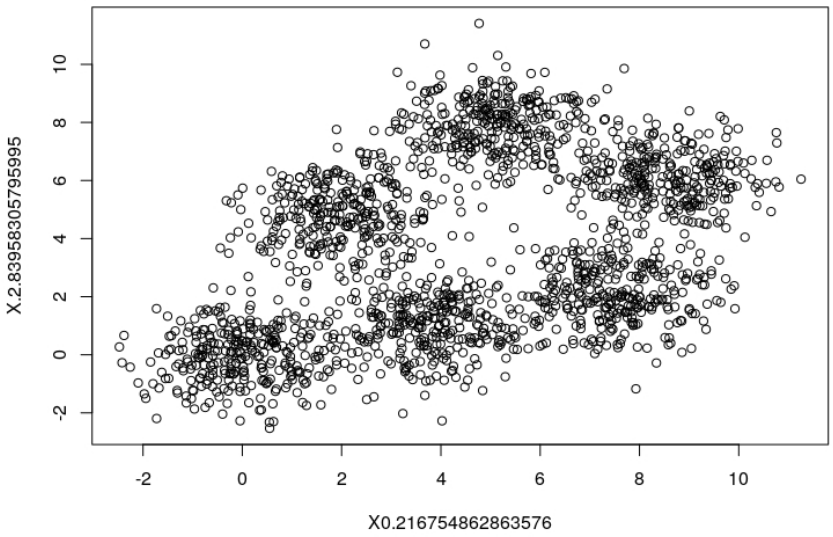
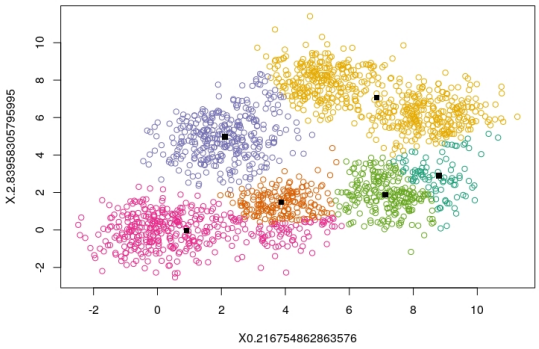
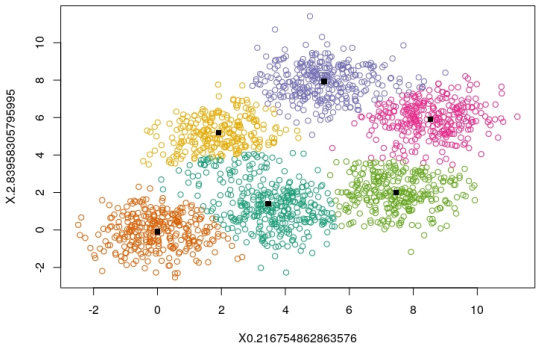


Figure 5: Dataset with 6 clusters partially overlap

Result



(a) K-means random initialization



(b) K-means++ initialization

3 Record audio and translate binary file to textual integers

1. I record 3 audios with sampling rate of 44100 Hz
2. I export them using the following options:
   - **Encoding:** Unsigned 8-bit PCM.
   - **Header:** RAW (header-less).

   I have 3 audios files:
   - audio1/audio1.raw
   - audio2/audio2.raw
   - audio3/audio3.raw
3. Translate the binary audio signal to a file with textual integers

```
./read audio1/audio1.raw > audio1/audio1.integer
./read audio2/audio2.raw > audio2/audio2.integer
./read audio3/audio3.raw > audio3/audio3.integer
```

4 Apply k-means on the file with textual integers

I use k-means++ to codify the audio signal.

4.1 audio1

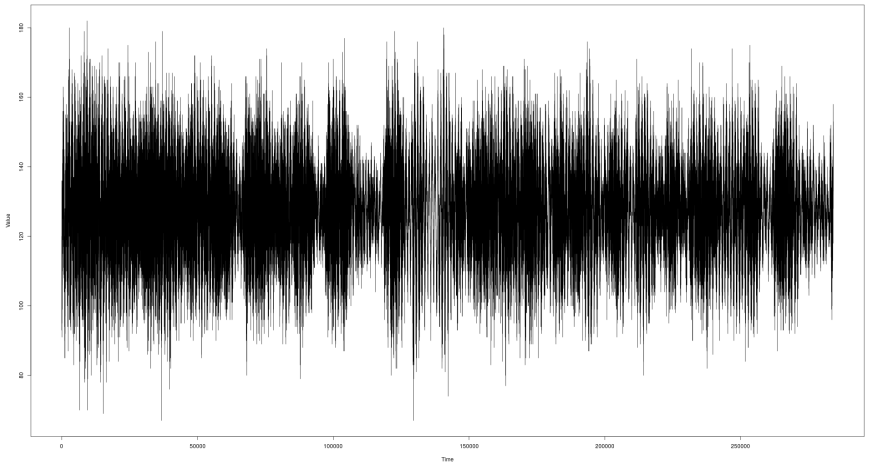
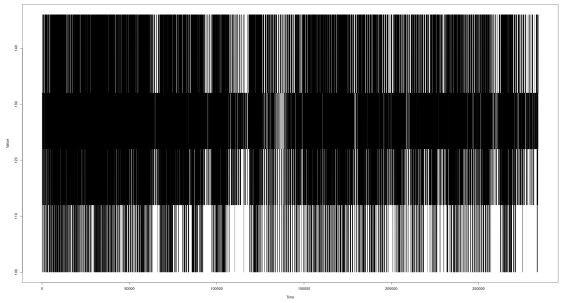
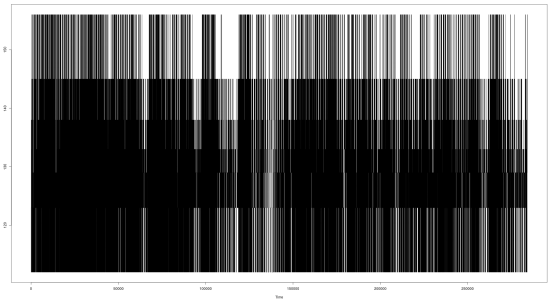


Figure 7: Original signal

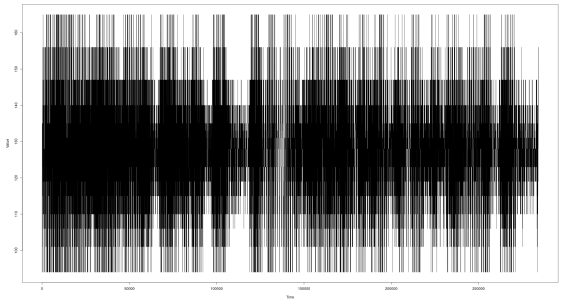
After running k-means



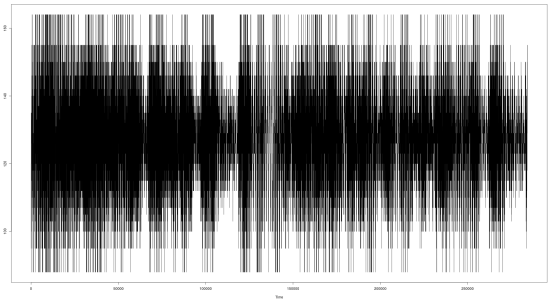
(a) k-means with k = 5



(b) k-means with k = 7



(c) k-means with k = 15



(d) k-means with k = 25

4.2 audio2

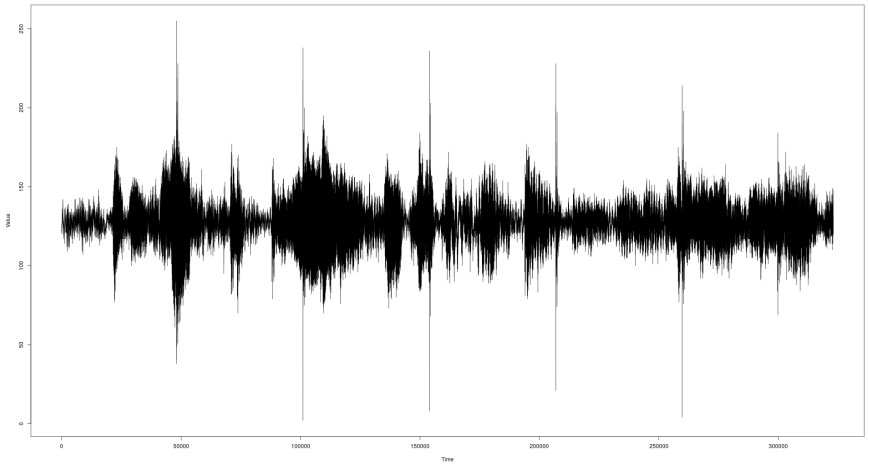
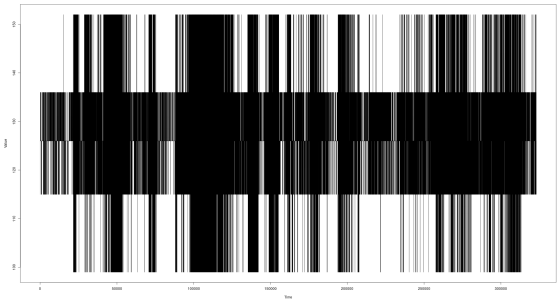
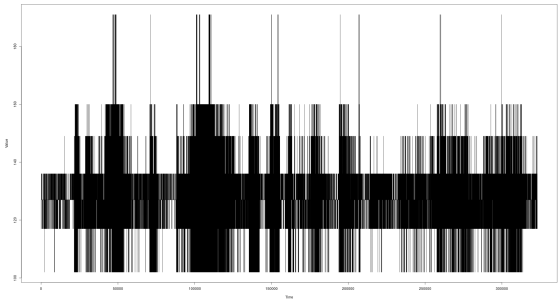


Figure 9: Original signal

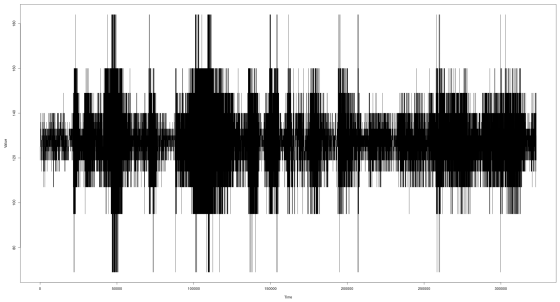
After running k-means



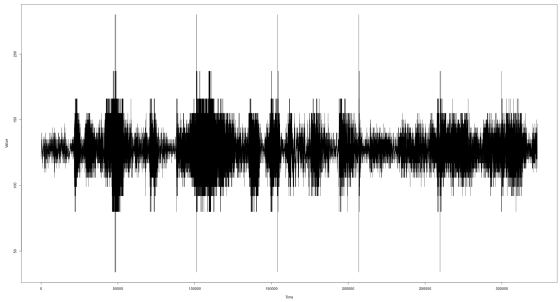
(a) k-means with k = 5



(b) k-means with k = 7



(c) k-means with k = 15



(d) k-means with k = 25

4.3 audio3

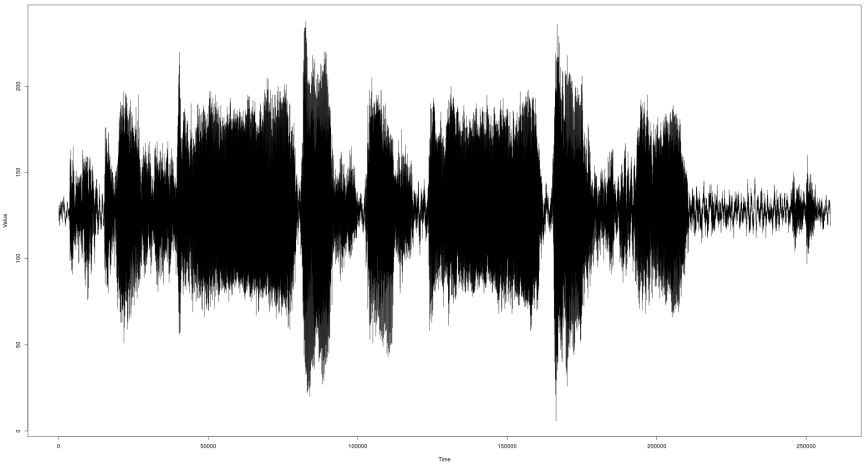
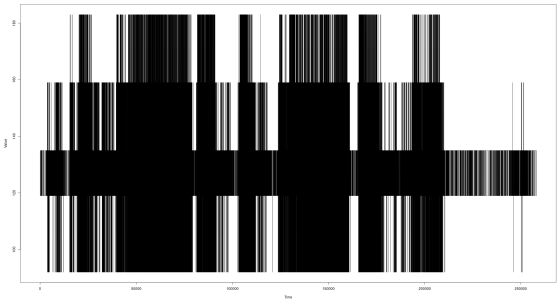
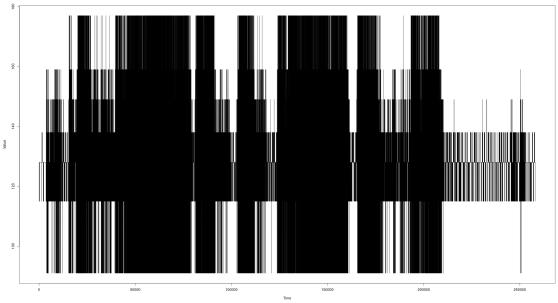


Figure 11: Original signal

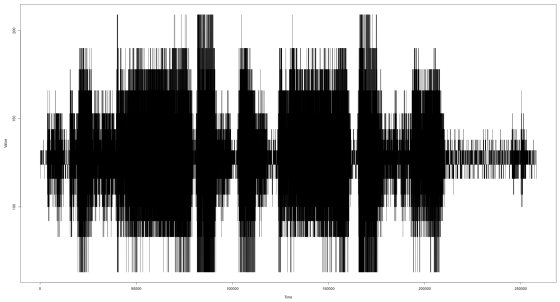
After running k-means



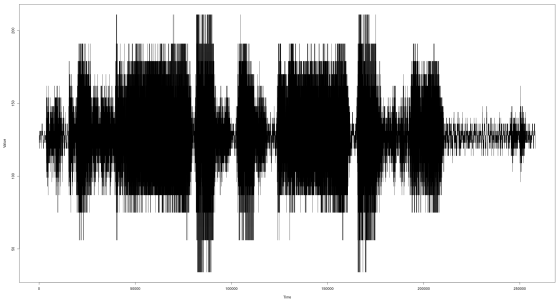
(a) k-means with k = 5



(b) k-means with k = 7



(c) k-means with k = 15



(d) k-means with k = 25

5 Translate and play the new audio files

1. Translate the textual integers file to binary file

```
./write audio3/k25/k25.integer > /dev/null
./write audio3/k15/k15.integer > /dev/null
./write audio3/k7/k7.integer > /dev/null
./write audio3/k5/k5.integer > /dev/null
```

2. Play the binary file

```
aplay -t raw -r 44100 audio3/k25/k25.raw
aplay -t raw -r 44100 audio3/k15/k15.raw
aplay -t raw -r 44100 audio3/k7/k7.raw
aplay -t raw -r 44100 audio3/k5/k5.raw
```

We do the same for audio1 and audio2.

We can hear that the quality of the audio decreases as we decrease the number of clusters because each value is change to the value of its cluster so that with 25 clusters we only have 25 values to represent the audio instead of 256 value in the original audio.

6 Test the compression

I use the **BZip2** for compression.

```
bzip2 -k k25/k25.raw
```

Compression Rate

- **audio1**: Size of original file after compression: **140.1 kB**
  - k = 5: 42.9 kB / 140.1 kB = 0.3
  - k = 7: 57.8 kB / 140.1 kB = 0.4
  - k = 15: 77.8 kB / 140.1 kB = 0.55
  - k = 25: 96.9 kB / 140.1 kB = 0.7
- **audio2**: Size of original file after compression: **133.1 kB**
  - k = 5: 31.4 kB / 133.1 kB = 0.24
  - k = 7: 34.8 kB / 133.1 kB = 0.26
  - k = 15: 69.1 kB / 133.1 kB = 0.52
  - k = 25: 84.3 kB / 133.1 kB = 0.63
- **audio3**: Size of original file after compression: **128.6 kB**
  - k = 5: 25.1 kB / 128.6 kB = 0.19
  - k = 7: 34.8 kB / 128.6 kB = 0.27
  - k = 15: 48.8 kB / 128.6 kB = 0.38
  - k = 25: 65.8 kB / 128.6 kB = 0.51